

일반 Task Graph MEC Offloading

Offloading and Resource Allocation with General Task Graph in Mobile Edge Computing: A Deep Reinforcement Learning Approach

Jia Yan, Suzhi Bi, Ying-Jun Angela Zhang (2020)

DNN이 적은 location 후보를 만들고, 최적화 기반 evaluator가 CPU 자원까지 포함한 ETC를 고른다

왜 task 하나씩 독립적으로 offload할 수 없을까

child task는 parent의 결과 data를 받아야 하므로, 한 task의 location이 다음 task의 ready time을 바꾼다

01	DAG 입력	vertex는 compute task edge는 data dependency
↓		
02	location 선택	각 task를 local 또는 edge에 둔다
↓		
03	boundary 발생	서로 다른 location이면 uplink 또는 downlink
↓		
04	critical path	가장 늦은 dependency path가 application 종료를 결정

일반 graph에서는 task별 빠른 선택이 아니라 dependency와 transfer를 함께 비교해야 한다

location 경계가 uplink와 downlink를 만든다

a_i 는 task i 전체를 local 또는 edge에 배치하는 binary 변수다



$a_i=0$ 은 local, $a_i=1$ 은 edge이며 partial execution은 이 논문의 action이 아니다

device energy와 종료시간을 함께 최소화한다

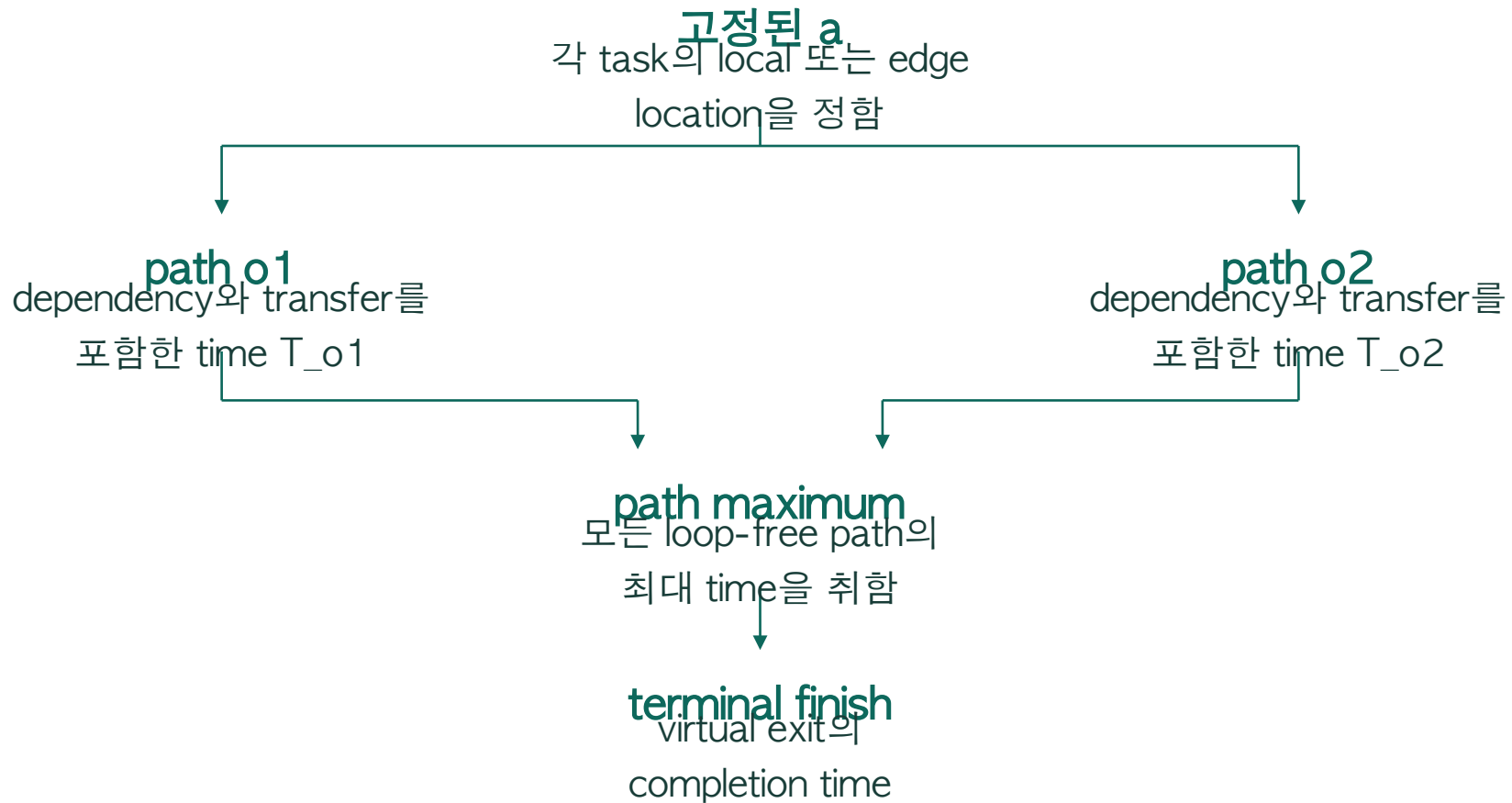
ETC는 MD가 쓰는 energy와 virtual exit가 local에서 끝나는 time의 가중합이다

항	무엇을 세나	location의 영향	주의
local compute time	$L_i / \text{local CPU}$	local task일 때	frequency가 클수록 짧음
local energy	$\text{kappa } L_i f_i^2$	local task일 때	frequency가 클수록 큼
transfer	uplink 또는 downlink	location boundary	dependency data에 따라 발생
ETC	$\text{beta}_e E + \text{beta}_t \text{ finish time}$	모든 decision	$\text{beta}_e + \text{beta}_t = 1$

모든 task를 edge로 보내는 선택도, 모두 local에 두는 선택도 항상 ETC 최소가 아니다

고정 action의 종료시간은 가장 늦은 path가 만든다

task의 location a를 먼저 고정하면, local CPU frequency allocation을 풀 수 있다



critic은 학습 network가 아니라 optimizer다

candidate a마다 Algorithm 1 이 local CPU frequency를 최적화하고 ETC를 계산한다

01	candidate a	binary location vector를 하나 고정
↓		
02	P2 구성	path maximum과 frequency constraint를 반영
↓		
03	dual update	subgradient로 multiplier를 갱신
↓		
04	ETC 평가	optimal frequency와 eta star를 반환

DNN이 ETC value를 근사하도록 critic을 학습하는 방식과 구별해야 한다

actor는 state에서 relaxed location vector를 낸다

output은 final binary action이 아니라 M개 task별 0과 1 사이의 score다



continuous score를 바로 실행하지 않고, GNOP로 제한된 binary candidate set을 만든다

GNOP는 순서를 보존해 binary 후보를 만든다

정확한 2의 M제곱 search 대신 actor가 제안한 B개만 evaluator가 검사한다

01	first candidate a1	각 score를 0.5에서 threshold
↓		
02	ordinary B/2	0.5와 거리 순으로 threshold를 이동
↓		
03	noise B/2	noise와 sigmoid 뒤 같은 quantization
↓		
04	Omega_t	B개의 binary vector를 evaluator로 전달

noise의 역할은 후보 다양화이며, noise가 ETC를 직접 최소화하는 것은 아니다

best candidate가 replay label이 된다

학습 target은 return이 아니라 같은 state에서 evaluator가 고른 binary action이다

01	candidate evaluation	각 a_b 의 eta star를 Algorithm 1로 계산
↓		
02	selection	최소 ETC candidate a_t star 선택
↓		
03	replay memory	state와 a_t star를 limited memory에 저장
↓		
04	cross-entropy	batch를 Adam으로 학습 periodic update

이 update는 optimization-selected action을 모방하도록 actor를 학습시키는 절차다

one-climb: path별 MD에서 edge로 한 번만 이동

candidate filter가 evaluator 호출 수를 줄이지만, 일반 graph 전체에서 무조건 optimal이라는 뜻은 아니다

path location pattern	MD to edge 이동	filter	source의 qualification
local -> edge	1 회	허용	one-climb definition
local -> edge -> local	1 회	허용	return은 허용
local -> edge -> local -> edge	2 회	제거	two-time offloading
overlapping paths	path마다 다름	주의	추가 mild condition 필요

one-climb은 speed heuristic이다. 전체 DAG optimality는 조건부다

구성 예시: parallel parent는 maximum으로 합친다

toy DAG start -> A,B -> C -> exit에서 resource를 고정해 completion-time 계산만 보인다

candidate	A/B branch ready	C와 exit	completion time
all local	$\max(4.0, 2.0) = 4.0$	C local 6.0	10.0 time unit
all edge	$\max(0.2+0.8, 0.2+0.4) = 1.0$	C edge 1.2 + return 0.2	2.4 time unit
what changes	location boundary adds transfer	evaluator would also optimize CPU	ETC needs energy too

이 toy는 parallel maximum만 보이며, reported ETC나 energy result가 아니다

한 epoch의 hybrid decision loop

search 공간을 줄이는 learner와 feasibility·ETC를 보장하는 optimizer가 역할을 나눈다



learner는 후보를, optimizer는 B개 후보의 ETC 평가를 맡는다

수치 실험 : 8-task graph와 wireless setting

아래 값은 algorithm의 보편 parameter가 아니라 source simulation condition이다

구성	source setting	학습 setting	비교 기준
graph	mesh, tree, general	각 graph 8 actual tasks	동일 workload vector
wireless	2 MHz, Rician fading	edge CPU U(2,50) GHz	single MD, single AP
objective	beta_e=beta_t=.5	f_peak=.01 GHz	energy-time weighted ETC
actor	hidden 160/120/80	lr .01, batch 128	memory 1024, interval 10

accuracy와 runtime은 source의 8-task simulation condition에 한정된다

학습 지표를 정확히 읽는 법

loss와 accuracy를 deployment quality나 universally optimal policy로 해석하면 안 된다

관측	source figure	조건	바르게 읽기
learning-rate sensitivity	Fig. 7a	.1, .01, .001 비교	.01이 이 setting에서 더 안정적
loss	Fig. 8	lr .01, batch 128, memory 1024	300 step 뒤 moving average < .1
accuracy chi	Fig. 9, Table II	50 independent realizations	exhaustive ETC와 비교한 ratio
general graph	Table II	동일 simulation condition	chi 99.9 percent

chi는 exhaustive ETC 대비 objective accuracy이며 task-label accuracy가 아니다

ETC comparison: adaptive 후보 선택의 효과

Fig. 10 compares DRL variants with Gibbs, exhaustive search, all-edge, and all-local after a warm-up

method	candidate/search idea	Fig. 10 finding	condition
DRL with GNOP	B binary candidates	near exhaustive and Gibbs	first 20,000 epochs excluded
DRL with one-climb	GNOP plus path filter	ETC hardly affected	heuristic qualification applies
Gibbs / exhaustive	iterative / all 2^M actions	reference methods	50 realization average
all edge / all local	fixed location	higher ETC in figure	no channel adaptation

source는 simulation에서 near-optimal ETC를 보였다. 모든 general graph 보장은 아니다

candidate filter가 source 계산시간을 낮춘다

Table III reports average time per realization with $B=16$, not an end-to-end production latency SLA

method	mesh	tree	general
one-climb GNOP	0.9240 s	1.3421 s	1.0464 s
GNOP	1.4702 s	1.4107 s	1.5821 s
Gibbs sampling	8.2039 s	8.3046 s	8.6101 s
exhaustive search	25.6690 s	26.8181 s	27.5185 s

general graph one-climb GNOP은 1.0464 s였다. source는 Gibbs보다 약 8배 짧다고 요약한다

source가 보인 것과 보이지 않은 것

hybrid candidate-search pattern은 유용하지만, model assumptions가 결과의 범위를 정한다

established in source	not established by source
fixed action에서 path-aware CPU allocation을 풀 수 있음	arbitrary workflow or partial-task execution
DNN proposal와 optimizer evaluation을 결합함	learned Q critic 또는 policy-gradient method
8-task simulation에서 near-optimal ETC evidence	multi-user, real deployment, arbitrary graph guarantee
one-climb으로 candidate evaluation을 줄임	overlapping DAG에서 unconditional optimality

핵심 contribution은 candidate proposal와 model-based evaluation의 결합이다

핵심 정리

일반 task graph MEC에서는 route, transfer, critical path, and CPU resource allocation이 한 decision problem이다

- task location은 dependency transfer와 child ready time을 바꾼다
- fixed location 뒤에는 continuous CPU allocation을 푼다
- GNOP가 candidate set을 만들고 optimizer가 ETC로 고른다
- replay update는 actor가 selected action을 더 잘 제안하게 한다
- reported result는 8-task source simulation condition에 한정된다

DNN 단독 solver가 아니라 candidate proposal와 model evaluation의 결합이다